

Supplementary Material: Additional Results and Audits

(Anonymous Code Snapshot)

Scope. This document provides additional tables and lightweight audits accompanying the anonymous code snapshot. All main reported metrics (e.g. CH@100, SH@100, and proof utility at RR64) follow the exact accounting used in the main paper: **generator-call budget** under strict debiting (seedbank reuse + patch/repair count toward the call budget).

1 Statement Generation at $T=100$ Calls (Main Table)

Table 1: Summary at $T = 100$ generator calls per problem. We report CH@100, SH@100, and cross-problem concentration of semantic-success counts (Gini and top-10% share over per-problem `semantic_ok` counts).

Variant	ProofNet (test, $N = 186$)				CombiBench ($N = 100$)			
	CH@100	SH@100	Gini↓	Top-10%↓	CH@100	SH@100	Gini↓	Top-10%↓
Sampling baselines (Kimina; no archive)								
Sample	0.903	0.715	0.537	0.243	1.000	0.440	0.816	0.637
Compile Repair	0.909	0.720	0.566	0.263	0.990	0.400	0.825	0.641
Compile+Semantic Repair	0.909	0.780	0.555	0.264	0.990	0.460	0.813	0.609
Hybrid sampling baseline (no archive): Kimina gen + Qwen3-30B-A3B repair								
Compile+Semantic Repair	0.973	0.828	0.505	0.241	1.000	0.530	0.790	0.588
Sampling baselines (no archive): Qwen3-30B-A3B gen + repair								
Sample	0.387	0.242	0.896	0.876	0.300	0.200	0.939	0.952
Compile Repair	0.371	0.237	0.912	0.911	0.900	0.260	0.740	0.385
Compile+Semantic Repair	0.688	0.548	0.802	0.656	0.940	0.390	0.827	0.655
FormalEvolve (ours)								
Ours (K=2)	0.973	0.849	0.443	0.229	1.000	0.580	0.759	0.531
Ours (K=1)	0.984	0.866	0.362	0.198	1.000	0.550	0.726	0.442
Ours (K=2; no EvolAST)	0.973	0.871	0.454	0.236	1.000	0.500	0.776	0.577
Ours (K=2; no patch repair)	0.903	0.780	0.449	0.211	1.000	0.470	0.772	0.494

2 Downstream Proof Utility (RR64, $B=64$)

Table 2: Downstream proof utility at $B = 64$ prover attempts per problem on the semantically consistent repertoire produced within $T = 100$ generator calls. We report solved-problem counts (x/N) for pass@64, complete@64, and theorem-complete@64.

Method	pass@64	complete@64	theorem-complete@64	ProofNet (test, $N = 186$)		
				pass@64	complete@64	theorem-complete@64
Ours (K=2)	44/100	27/100	13/100	127/186	52/186	45/186
Sample	41/100	23/100	8/100	106/186	46/186	41/186
Compile+Semantic Repair (Kimina)	40/100	23/100	8/100	119/186	50/186	46/186

3 Patch/Repair Call Breakdown (Audit)

Notes. These audits summarize the logged patch/repair calls by patch type for the patch/repair model. Counts include only calls that logged token usage.

Table 3: Patch/repair call breakdown by patch type (CombiBench).

Variant	full	diff	cross	semantic_repair
FormalEvolve (main)	643	677	376	141
FormalEvolve (w/o EvolAST)	531	484	419	101
FormalEvolve (w/o repair)	1436	1138	698	0

Table 4: Patch/repair call breakdown by patch type (ProofNet).

Variant	full	diff	cross	semantic_repair
FormalEvolve (main)	3075	2361	1251	359
FormalEvolve (w/o EvolAST)	3124	2523	1335	328
FormalEvolve (w/o repair)	4349	3332	1693	0

4 Token Usage Audit (Patch/Repair Model Only)

Notes. Token usage is reported from meta logs for the patch/repair model calls only. Seedbank reuse is debited in call budgets but does not appear as a live LLM call in these token logs.

Table 5: Generator token usage from meta logs (CombiBench; patch/repair model calls only).

Variant	Calls(w/ tokens)	InputTok	OutputTok	TotalTok	AvgTok/call
FormalEvolve (main)	3106	2.88M	0.40M	3.28M	1055.2
FormalEvolve (w/o EvolAST)	1535	1.08M	0.21M	1.29M	843.1
FormalEvolve (w/o repair)	7745	7.18M	1.04M	8.22M	1061.4

5 Additional Study: ReForm-as-Seeder (CombiBench100, @16)

Goal. This study treats a strong single-trajectory system (ReForm) *only* as a seed provider: we take its generated statements as the seed bank, and then run FormalEvolve as a plug-in handover mechanism under a strict call budget. The key question is not token-parity; instead we ask whether population-based compile-gated search reduces cross-problem concentration (lower Gini / lower top-10% share) under a tight call budget.

6 Wall-clock Timing (Optional Audit)

Caveat. Wall-clock times depend on machine/filesystem and any optional external services (e.g., Lean servers). These numbers are provided only as a lightweight audit and should not be interpreted as compute-matched comparisons.

Table 6: Generator token usage from meta logs (ProofNet; patch/repair model calls only).

Variant	Calls(w/ tokens)	InputTok	OutputTok	TotalTok	AvgTok/call
FormalEvolve (main)	7725	6.54M	0.73M	7.27M	941.4
FormalEvolve (w/o EvolAST)	7310	6.21M	0.97M	7.18M	981.7
FormalEvolve (w/o repair)	10932	9.28M	1.05M	10.33M	945.3

Table 7: ReForm-as-Seeder handover case study (external unified compile gate + judge). SemOK_{total} is the total number of semantic-ok candidates aggregated across problems (used to interpret uniformity vs “just fewer successes”).

Method	Budget	CH@16	SH@16	SemOK _{total}	Gini↓	Top10%↓
ReForm	sample@8	96	91	338	0.4196	0.2337
ReForm	sample@16	97	93	672	0.4268	0.2247
Ours (ReForm seed)	calls@16, $k=4$	96	86	436	0.3841	0.1950
Ours (ReForm seed)	calls@16, $k=6$	97	91	497	0.3813	0.2032
Ours (ReForm seed)	calls@16, $k=8$	97	91	514	0.3912	0.2179
Ours (ReForm seed)	calls@16, $k=9$	96	91	545	0.4061	0.2220

Table 8: Per-problem wall-clock elapsed time from the run launcher (CombiBench), in hours.

Method	Median(h)	P90(h)	Max(h)
Sample	1.13	1.32	1.39
Compile Repair	1.29	6.82	7.08
Compile+Semantic Repair	1.32	1.71	1.97
FormalEvolve (main)	1.97	2.46	2.80
FormalEvolve (w/o EvolAST)	1.96	2.26	2.35
FormalEvolve (w/o repair)	3.22	4.93	6.94

Table 9: Per-problem wall-clock elapsed time from the run launcher (ProofNet), in hours.

Method	Median(h)	P90(h)	Max(h)
Sample	1.58	1.71	1.99
Compile Repair	1.70	2.26	2.52
Compile+Semantic Repair	1.75	2.26	2.69
FormalEvolve (main)	0.42	0.65	0.78
FormalEvolve (w/o EvolAST)	0.64	0.82	2.26
FormalEvolve (w/o repair)	0.40	0.63	1.88